

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

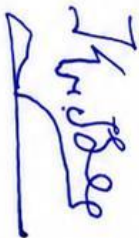
Scenario based Questions

Code of Conduct:

I K DHANUSH SRIVATSAN / 192524503 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

Question 1 – CPU Scheduling in a Real-Time Railway Ticket Booking System

Scenario Overview

A railway ticket booking system is a real-time application that processes thousands of booking requests every day. During festival seasons and holidays, the number of users accessing the system increases significantly. The server must execute several processes simultaneously, including **payment processing (P1)**, **seat availability checking (P2)**, and **ticket confirmation (P3)**. Since these processes compete for CPU resources, the operating system must decide which process should be executed first.

The burst time of these processes varies between **2 ms and 20 ms**. Customers have reported delays in ticket confirmation during peak hours, indicating that the current CPU scheduling method is not efficient. Therefore, selecting an appropriate CPU scheduling algorithm is essential to improve system performance, reduce waiting time, and enhance customer satisfaction.

Problem Analysis

The main challenge in this scenario is that different processes have different levels of importance. Payment processing is the most critical operation because any delay may result in failed or duplicate transactions. Seat availability checking is also important because customers need immediate information before booking a ticket. Ticket confirmation is the final stage of the booking process and can tolerate a slight delay compared to payment processing.

If the operating system schedules these processes without considering their priority, critical tasks may wait behind less important tasks. This increases waiting time, response time, and reduces the overall efficiency of the system.

Comparison of Scheduling Algorithms

Different CPU scheduling algorithms can be used in this scenario.

First Come First Serve (FCFS)

FCFS executes processes in the order of arrival. Although it is simple to implement, it does not consider process priority. During heavy traffic, important payment transactions may wait behind less important processes, increasing response time.

Shortest Job First (SJF)

SJF executes the process with the shortest burst time first. It reduces average waiting time but ignores process priority. A short ticket confirmation task may execute before a payment transaction, making it unsuitable for real-time applications.

Round Robin (RR)

Round Robin allocates equal CPU time to every process using a fixed time quantum. It provides fairness but generates frequent context switching and does not prioritize urgent processes.

Priority Scheduling

Priority Scheduling executes the highest-priority process first. Since payment processing is the most critical task, this algorithm ensures that important operations receive immediate CPU access.

Process Operation		Priority
P1	Payment Gateway Transaction	High
P2	Seat Availability Check	Medium
P3	Ticket Confirmation	Low

Among these algorithms, **Priority Scheduling** is the most suitable for this application.

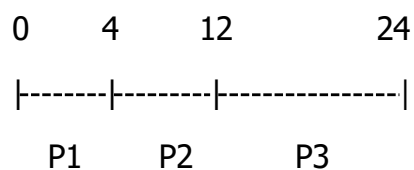
Recommended Scheduling Algorithm

The recommended algorithm is **Priority Scheduling** because it allocates CPU time according to the importance of each process.

The execution order is:

P1 → P2 → P3

Sample Gantt Chart



This sequence ensures that payment transactions are completed first, followed by seat availability checking and ticket confirmation. As a result, critical tasks are completed without unnecessary delays.

Justification

Priority Scheduling offers several advantages in the railway ticket booking system:

- Reduces response time for payment transactions.

- Decreases waiting time for critical processes.
- Improves CPU utilization by executing important ready processes first.
- Increases system throughput during peak booking periods.
- Enhances customer satisfaction by reducing booking delays.
- Ensures efficient handling of high-priority operations.

Compared with FCFS, SJF, and Round Robin, Priority Scheduling provides better performance for real-time applications where different processes have different levels of importance.

Limitation and Solution

The main limitation of Priority Scheduling is **starvation**. If high-priority processes continue arriving, low-priority processes such as ticket confirmation may wait for a long time.

This problem can be solved using the **Aging** technique. Aging gradually increases the priority of waiting processes, ensuring that every process eventually gets CPU time. This improves fairness without affecting the execution of critical tasks.

Conclusion

The railway ticket booking system requires an efficient CPU scheduling algorithm to manage multiple processes during peak booking periods. After comparing FCFS, SJF, Round Robin, and Priority Scheduling, **Priority Scheduling** is identified as the most suitable solution because it prioritizes payment transactions, reduces response time, improves CPU utilization, and increases system throughput. Although starvation is a limitation, it can be effectively prevented using the Aging technique. Therefore, Priority Scheduling provides the best balance between performance, efficiency, and fairness for a real-time railway ticket booking system.

Question 2 – Deadlock in a Banking Transaction System

Scenario Overview

A banking transaction system processes thousands of customer requests every day, including money transfers, passbook updates, balance inquiries, and statement generation. To execute these transactions successfully, processes require access to

shared resources such as the **Account Database (A)**, **Customer Information Database (B)**, and **Transaction Log (C)**. Since these resources are limited, the operating system must allocate them efficiently.

In the given scenario:

- **T1** holds **Resource A** and requests **Resource B**.
- **T2** holds **Resource B** and requests **Resource C**.
- **T3** holds **Resource C** and requests **Resource A**.

Each transaction is waiting for another transaction to release a resource, preventing further execution. This situation can interrupt banking operations and affect customer service.

Problem Analysis

The major issue in this scenario is **deadlock**. A deadlock occurs when two or more processes wait indefinitely for resources held by one another, causing none of them to proceed.

Here:

- T1 cannot continue because it is waiting for Resource B.
- T2 cannot continue because it is waiting for Resource C.
- T3 cannot continue because it is waiting for Resource A.

Since every required resource is already occupied, all three transactions remain blocked. As a result, the banking server cannot process any of these transactions, leading to increased waiting time, delayed services, and reduced system performance.

Deadlock Analysis

The resource dependency can be represented as follows:

T1 → Waiting for Resource B

T2 → Waiting for Resource C

T3 → Waiting for Resource A

$T1 \leftarrow T3 \leftarrow T2$

This forms a **circular waiting chain**, which confirms the existence of a deadlock.

A deadlock occurs only when the following four conditions are satisfied:

Mutual Exclusion

Each database resource can be accessed by only one transaction at a time. Other transactions must wait until the resource is released.

Hold and Wait

Each transaction already holds one resource while requesting another resource.

No Preemption

The operating system cannot forcibly remove a resource from a transaction. The resource is released only after the transaction completes.

Circular Wait

A circular dependency exists because T1 waits for T2, T2 waits for T3, and T3 waits for T1.

Since all four conditions are present, the banking system is in a deadlock state.

Recommended Deadlock Handling Technique

The most suitable technique for this banking system is **Deadlock Prevention**.

Deadlock Prevention avoids deadlocks by eliminating one of the four necessary conditions. The best approach is to eliminate the **Circular Wait** condition by forcing every transaction to request resources in the same order.

The operating system should define the following resource allocation sequence:

Account Database (A)

↓

Customer Information Database (B)

↓

Transaction Log (C)

Every transaction must request resources only in this order. This prevents circular waiting and ensures smooth execution of banking operations.

Improved Resource Allocation Strategy

The improved execution sequence is shown below:

Lock Resource A

↓

Lock Resource B

↓

Lock Resource C

↓

Execute Transaction

↓

Release Resource C

↓

Release Resource B

↓

Release Resource A

This strategy guarantees that resources are allocated consistently, preventing deadlocks and improving overall system stability.

Benefits and Limitation

Benefits

Implementing Deadlock Prevention provides several advantages:

- Eliminates deadlocks before they occur.
- Improves transaction reliability.
- Reduces waiting time.
- Increases CPU utilization.
- Enhances throughput.
- Ensures smooth banking services.

Limitation

A limitation of Deadlock Prevention is that transactions may wait longer to obtain resources, resulting in slightly lower resource utilization. However, this trade-off is acceptable because maintaining correct and reliable banking transactions is more important than maximizing resource usage.

Conclusion

The banking transaction system experiences a deadlock because three transactions hold different resources while waiting for resources held by one another. Analysis shows that all four necessary deadlock conditions—Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait—are satisfied. Among the available solutions, **Deadlock Prevention** is the most appropriate because it removes the possibility of circular waiting by enforcing a fixed resource allocation order. This approach improves CPU utilization, enhances system reliability, reduces waiting time, and ensures secure and uninterrupted banking operations.

Question 3 – Mobile Operating System App Switching Lag

Scenario Overview

Modern smartphones support multitasking by allowing multiple applications to run simultaneously. Applications such as WhatsApp, music players, web browsers, and social media apps execute concurrently while sharing CPU resources. The operating system is responsible for scheduling these applications efficiently to provide a smooth user experience.

In the given scenario, users experience delays while switching between applications. The background music player slows down when WhatsApp is opened, application switching becomes laggy, and the CPU shows high idle time even though nearly **20 background applications** are active. The current operating system uses the **Round Robin scheduling algorithm** with a **20 ms time quantum**. These symptoms indicate that the existing scheduling policy is not efficiently managing CPU resources.

Problem Analysis

The primary issue is inefficient CPU scheduling during multitasking. Although many applications are running, most of them spend a significant amount of time waiting for **network communication, storage access, or user input** instead of actively using the CPU.

For example:

- WhatsApp waits for internet messages.
- Music players wait for audio buffering.
- Browsers wait for webpage downloads.
- Social media applications wait for server responses.

When an application waits for I/O operations, it enters the **Waiting (Blocked) State**, allowing the operating system to execute another ready process. However, if many applications are blocked simultaneously, the CPU may become idle even though several applications remain open. This leads to reduced CPU utilization and slower application switching.

Analysis of the Current Scheduling Algorithm

The operating system currently uses **Round Robin Scheduling** with a **20 ms time quantum**.

Advantages

- Provides equal CPU time to every application.
- Suitable for multitasking environments.

- Prevents starvation by ensuring every process gets CPU access.

Disadvantages

- A 20 ms time quantum may increase response time for interactive applications.
- Frequent context switching introduces scheduling overhead.
- High-priority applications receive the same CPU time as background applications.
- User experience may degrade during heavy multitasking.

Although Round Robin provides fairness, it does not efficiently prioritize foreground applications.

Recommended Scheduling Technique

The most suitable scheduling technique is **Priority-Based Round Robin Scheduling**.

In this approach:

- Foreground applications such as WhatsApp and active browsers are assigned **high priority**.
- Background applications such as music players and synchronization services receive **lower priority**.
- CPU time is still shared using Round Robin, but important applications are executed first.

The execution sequence is shown below:

Foreground Applications

↓

Priority Queue

↓

Round Robin Scheduling

↓

Background Applications

↓

CPU Execution

This scheduling method ensures that active applications receive faster CPU access while background tasks continue executing fairly.

Performance Improvement

Implementing Priority-Based Round Robin improves system performance in several ways.

- Reduces application switching delay.
- Improves response time for active applications.
- Increases CPU utilization by efficiently scheduling ready processes.
- Enhances multitasking performance.
- Improves user experience during heavy workloads.
- Reduces waiting time for foreground applications.
- Maintains fairness among background applications.

Compared to the existing scheduling policy, the proposed approach provides better responsiveness without significantly increasing CPU overhead.

Limitation and Solution

A limitation of Priority-Based Round Robin is that assigning higher priority to foreground applications may delay background tasks if not managed properly.

This issue can be minimized by implementing the **Aging** technique. Aging gradually increases the priority of waiting background processes, ensuring that every application eventually receives CPU time while maintaining system fairness.

Conclusion

The mobile operating system experiences application switching delays because many background applications spend significant time waiting for I/O operations, resulting in inefficient CPU scheduling. Although Round Robin Scheduling provides fairness, its 20 ms time quantum is not ideal for interactive mobile applications. **Priority-Based Round Robin Scheduling** is the most suitable solution because it prioritizes active applications, reduces response time, improves CPU utilization, enhances multitasking performance, and provides a smoother user experience. By combining this approach with the Aging technique, the operating system can achieve both fairness and high performance during multitasking.

Question 4 – Deadlock Risk During an Operating System Update

Scenario Overview

Operating system updates are essential for improving system security, fixing software bugs, and enhancing overall performance. During an update, multiple modules execute simultaneously and require access to shared resources such as **Disk I/O (R1)**, **Kernel Lock (R2)**, and **Configuration Lock (R3)**. Since these resources are limited, the operating system must allocate them efficiently to ensure successful completion of the update process.

In the given scenario:

- **M1 (Copy Update Files)** holds **R1** and requests **R2**.
- **M2 (Install Drivers)** holds **R2** and requests **R3**.
- **M3 (Update Registry)** holds **R3** and requests **R1**.

This resource allocation creates a dependency among the modules, preventing them from completing their tasks. As a result, the operating system update may stop unexpectedly, affecting system stability and reliability.

Problem Analysis

The major issue in this scenario is the occurrence of a **deadlock**. A deadlock occurs when two or more processes or modules are permanently blocked because each is waiting for a resource held by another.

In this case:

- M1 cannot proceed because it is waiting for R2.
- M2 cannot proceed because it is waiting for R3.
- M3 cannot proceed because it is waiting for R1.

Since every required resource is occupied, none of the modules can continue execution. This delays the update process and may cause update failures, system crashes, or incomplete software installation.

Deadlock Analysis

The relationship between modules and resources can be represented as follows:

M1 → Waiting for R2

M2 → Waiting for R3

M3 → Waiting for R1

M1 ← M3 ← M2

This forms a **circular waiting chain**, confirming that a deadlock exists.

According to operating system principles, four necessary conditions must be satisfied for a deadlock to occur.

Mutual Exclusion

Each resource can be allocated to only one module at a time. Other modules must wait until the resource is released.

Hold and Wait

Each module already holds one resource while requesting another resource.

No Preemption

The operating system cannot forcibly remove a resource from a module. Resources are released only after the module completes execution.

Circular Wait

A circular dependency exists because M1 waits for M2, M2 waits for M3, and M3 waits for M1.

Since all four conditions are present, the operating system is in a deadlock state.

Recommended Operating System Technique

The most suitable technique to prevent this problem is **Banker's Algorithm**.

Banker's Algorithm is a deadlock avoidance algorithm that allocates resources only if the system remains in a **safe state**. Before granting a resource request, the operating system checks whether all modules can complete execution successfully. If granting the request leads to an unsafe state, the request is postponed until resources become available.

A safe execution sequence for this scenario is:

M1 → M2 → M3 → M4

Using this sequence, each module completes its task and releases its resources before the next module begins execution, preventing deadlock.

Improved System Configuration

The improved execution strategy is shown below.

Allocate Resources



Execute M1



Release Resources



Execute M2



Release Resources



Execute M3



Release Resources



Execute M4



Update Completed

This configuration ensures that resources are allocated in a controlled manner, preventing circular waiting and allowing the operating system update to finish successfully.

Benefits and Limitation

Benefits

Implementing Banker's Algorithm provides several advantages:

- Prevents deadlocks by maintaining a safe resource allocation state.
- Improves CPU utilization by reducing unnecessary waiting.
- Increases system reliability during updates.
- Reduces update failures.
- Improves overall system performance.
- Ensures smooth execution of update modules.

Limitation

The main limitation of Banker's Algorithm is that it requires the operating system to know the maximum resource requirement of every module in advance. It also increases computational overhead because the system must continuously evaluate whether each resource request keeps the system in a safe state.

Conclusion

The operating system update scenario leads to a deadlock because modules M1, M2, and M3 hold different resources while waiting for resources held by one another. Analysis confirms that all four necessary deadlock conditions are satisfied. **Banker's Algorithm** is the most appropriate solution because it allocates resources only when the system remains in a safe state, thereby preventing deadlocks. This approach improves CPU utilization, enhances system reliability, reduces update failures, and ensures successful completion of operating system updates.

Question 5 – Cloud Server CPU Utilization Analysis

Scenario Overview

Cloud computing enables multiple Virtual Machines (VMs) to run on a single physical server, allowing efficient utilization of hardware resources. Each VM performs different tasks such as web hosting, database management, file storage, and application execution. The operating system manages CPU allocation among these VMs to ensure maximum performance and resource utilization.

In the given scenario, the cloud hosting provider observes that **80% of the time VM processes are waiting for network or disk operations**, while only **20% of the time is spent performing CPU computations**. Each server hosts **7 Virtual Machines (VMs)**. Since many VM processes remain in the waiting state due to I/O operations, the operating system must efficiently schedule CPU resources to improve utilization and throughput.

The CPU utilization is calculated using the following formula:

$$[\\ U = 1 - p^n \\]$$

Where:

- **U** = CPU Utilization
- **p** = I/O wait probability
- **n** = Number of Virtual Machines

Given:

- **p = 0.8**
- **n = 7**

CPU Utilization Calculation

Using the formula:

$$[\\ U = 1 - p^n \\]$$

Substituting the given values:

$$[\\ U = 1 - (0.8)^7 \\]$$

Calculating,

$$[$$

$$0.8^7 = 0.2097$$

$$]$$

Therefore,

$$[$$

$$U = 1 - 0.2097$$

$$]$$

$$[$$

$$U = 0.7903$$

$$]$$

$$[$$

$$\boxed{\text{CPU Utilization} = 79.03\%}$$

$$]$$

Hence, the CPU utilization of the cloud server is approximately **79%**.

Interpretation of CPU Utilization

The calculated CPU utilization of **79.03%** indicates that the processor is busy most of the time, while about **21% of the CPU remains idle**. This means the server is **efficiently utilized**, but it is **not fully optimized** because many VM processes spend a significant amount of time waiting for I/O operations.

The server is **not underutilized**, since the CPU is active nearly 80% of the time. However, there is still room for improvement by reducing I/O wait time and optimizing process scheduling. A utilization level between **70% and 90%** is generally considered suitable because it provides good performance while allowing capacity for workload spikes.

Effect of Changing the Number of Virtual Machines

The number of Virtual Machines directly affects CPU utilization and throughput.

Increasing the Number of VMs

When more VMs are added:

- More processes become available for execution.
- CPU idle time decreases.
- CPU utilization increases.
- Throughput improves because more tasks are completed simultaneously.

However, adding too many VMs can cause:

- Frequent context switching.
- Increased memory usage.
- Resource contention.
- Reduced performance after the optimal limit.

Decreasing the Number of VMs

When the number of VMs is reduced:

- Fewer processes are available for execution.
- CPU remains idle more frequently during I/O operations.
- CPU utilization decreases.
- Throughput reduces because fewer tasks are processed at a time.

Therefore, an optimal number of VMs should be maintained to balance CPU utilization and system performance.

Operating System Strategies to Improve CPU Utilization

1. Multiprogramming

Multiprogramming allows several VM processes to remain in memory simultaneously. Whenever one process waits for network or disk operations, the operating system immediately switches the CPU to another ready process.

This strategy:

- Reduces CPU idle time.
- Improves CPU utilization.
- Increases throughput.
- Makes better use of system resources.

2. Efficient CPU Scheduling

The operating system should use an efficient scheduling algorithm such as **Multi-Level Feedback Queue (MLFQ)** or **Round Robin**.

These algorithms:

- Allocate CPU time efficiently.
- Improve response time.

- Reduce waiting time.
- Keep the CPU busy by scheduling ready processes.

Among these, **MLFQ** is more suitable for cloud environments because it dynamically adjusts process priorities based on workload.

Benefits and Limitation

Benefits

Implementing the proposed strategies provides several advantages:

- Higher CPU utilization.
- Reduced CPU idle time.
- Increased throughput.
- Better resource utilization.
- Improved response time.
- Enhanced overall cloud server performance.

Limitation

Increasing the number of Virtual Machines beyond the server's capacity can lead to excessive context switching, memory contention, and scheduling overhead. As a result, overall system performance may decrease despite having more VMs. Therefore, administrators should allocate an optimal number of VMs based on the available hardware resources.

Conclusion

The calculated CPU utilization of the cloud server is **79.03%**, indicating that the processor is efficiently utilized but still has scope for improvement. Increasing the number of Virtual Machines can improve throughput and CPU utilization up to an optimal level, while decreasing the number of VMs reduces system performance. To improve CPU utilization, the operating system should implement **Multiprogramming** and **Multi-Level Feedback Queue (MLFQ) Scheduling**, which reduce CPU idle time, improve resource utilization, increase throughput, and ensure efficient execution of cloud workloads.

Question 6 – CPU Scheduling Scenario

Scenario Overview

A large-scale organization operates multiple applications and services that execute simultaneously during peak working hours. Employees access databases, cloud applications, file servers, and enterprise software at the same time, resulting in a large number of processes competing for CPU resources. Recently, the organization has observed increased response time, reduced throughput, and performance degradation during busy periods. These problems indicate that the existing CPU scheduling strategy is unable to manage the workload efficiently. Therefore, the operating system must adopt an effective scheduling technique to improve CPU utilization and overall system performance.

Root Cause Analysis

The primary cause of the performance issue is the excessive number of processes competing for limited CPU resources. During peak usage, CPU-intensive and interactive processes enter the ready queue simultaneously, creating resource contention. Frequent context switching increases CPU overhead, while long-running processes occupy the processor for extended periods. As a result, interactive processes experience delays, the ready queue becomes overloaded, and system throughput decreases. Poor scheduling decisions further increase waiting time and reduce the responsiveness of the operating system.

Recommended Operating System Technique

The most appropriate scheduling algorithm for this scenario is the **Multi-Level Feedback Queue (MLFQ) Scheduling Algorithm**.

MLFQ maintains multiple queues with different priority levels. New and interactive processes are placed in the highest-priority queue so they receive immediate CPU access. CPU-intensive processes that consume longer execution time are gradually moved to lower-priority queues. The scheduler continuously adjusts process priorities based on their behavior, ensuring efficient CPU allocation and preventing starvation.

Justification of the Recommendation

MLFQ is suitable for this environment because it improves several important performance metrics.

- **CPU Utilization:** The CPU remains busy by executing the highest-priority ready process, reducing idle time.
- **Throughput:** More processes are completed within a given period because short and interactive processes finish quickly.

- **Response Time:** Interactive applications receive faster CPU access, improving user experience.
- **Fairness:** Processes waiting for a long time can be promoted using the Aging technique, ensuring that every process eventually receives CPU time.

Compared with FCFS, SJF, and Round Robin, MLFQ provides better adaptability and overall system performance in environments with mixed workloads.

Improved System Configuration

The operating system should organize processes into multiple priority queues as shown below.

New Processes

|



High Priority Queue

(Interactive Processes)

|



Medium Priority Queue

(Regular Processes)

|



Low Priority Queue

(CPU-Intensive Processes)

|



CPU Execution

|



Completed Processes

In this configuration:

- New processes enter the highest-priority queue.
- Interactive processes execute first.
- CPU-intensive processes move to lower-priority queues after using their time quantum.
- Aging increases the priority of waiting processes to prevent starvation.
- Resources are released immediately after process completion.

This execution sequence improves responsiveness while maintaining fairness.

Limitation of the Proposed Solution

Although Multi-Level Feedback Queue Scheduling offers excellent performance, it has certain limitations. The algorithm is more complex than traditional scheduling methods because it requires maintaining multiple queues and dynamically adjusting process priorities. Frequent priority changes also introduce additional scheduling overhead. However, these disadvantages are relatively small compared to the significant improvements in CPU utilization, throughput, response time, and fairness.

Conclusion

The performance issues in the organization's CPU scheduling environment are mainly caused by CPU contention, overloaded ready queues, and inefficient scheduling during peak usage. After analyzing the available scheduling techniques, **Multi-Level Feedback Queue (MLFQ)** is identified as the most suitable solution because it dynamically adjusts process priorities according to workload characteristics. This approach improves CPU utilization, increases throughput, reduces response time, and ensures fair CPU allocation among all processes. Although MLFQ introduces additional implementation complexity, its ability to efficiently manage large-scale workloads makes it the best scheduling strategy for the given scenario.

Question 7 – Process Management in an E-Commerce System

Scenario Overview

A leading international e-commerce company operates in **70 countries** and processes nearly **10 million customer requests every day**. The application performs several activities such as user authentication, product search, shopping cart updates, payment processing, inventory management, and shipment tracking. During festive sales, millions of customers access the platform simultaneously, causing increased response time and reduced system performance. The operating

system must efficiently manage these activities as processes while ensuring smooth execution through process scheduling, Process Control Blocks (PCB), context switching, and multiprogramming.

Analysis of Processes Involved

The operating system treats each activity as a separate process to enable concurrent execution.

The major processes involved are:

- **User Authentication Process** – Verifies user credentials during login.
- **Product Search Process** – Retrieves product details from the database.
- **Shopping Cart Process** – Updates and stores selected products.
- **Payment Processing Process** – Handles secure payment transactions.
- **Inventory Management Process** – Updates stock availability after purchases.
- **Shipment Tracking Process** – Provides delivery status to customers.

Since these processes execute simultaneously, the operating system schedules CPU time efficiently to maintain high performance and reduce waiting time.

Program vs Process

A **program** is a passive set of instructions stored on a storage device, whereas a **process** is an active instance of a program currently executing in memory.

For example:

- The **Payment Application** stored on the server is a **program**.
- When a customer makes an online payment, the executing payment application becomes a **process**.

Similarly, the product search software is a program, while each customer search request creates a separate process. Therefore, one program can create multiple processes depending on the number of users.

Role of Process Control Block (PCB)

The **Process Control Block (PCB)** is a data structure maintained by the operating system to manage every running process.

A PCB contains important information such as:

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Scheduling Information
- Memory Allocation Details
- I/O Status Information

Whenever the CPU switches from one process to another, the operating system saves the current process information in its PCB and restores the information of the next process. This enables smooth execution without losing process data.

Process States in Payment Transaction

A payment transaction passes through several process states during execution.

New



Ready



Running



Waiting (Payment Verification)



Ready



Running



Terminated

Initially, the payment process is created (**New**), waits in the **Ready** queue, executes in the **Running** state, enters the **Waiting** state while communicating with the payment gateway, returns to the **Ready** state after receiving a response, resumes execution, and finally reaches the **Terminated** state after successful payment.

Impact of Context Switching

Context switching occurs when the CPU changes from one running process to another.

During festive sales, thousands of customer requests execute simultaneously. Frequent context switching ensures that every process receives CPU time, improving responsiveness. However, excessive context switching increases CPU overhead because the operating system spends additional time saving and restoring PCB information. As a result, throughput decreases and response time increases if context switching becomes too frequent.

Recommendations for Better Process Management

To improve process management and scalability, the organization should implement the following strategies:

- Use **Multiprogramming** to keep the CPU busy while processes wait for I/O operations.
- Implement **Multi-Level Feedback Queue (MLFQ) Scheduling** to prioritize interactive customer requests.
- Optimize context switching to reduce scheduling overhead.
- Use load balancing to distribute workloads across multiple servers.
- Implement efficient memory management to support a large number of concurrent processes.

These strategies improve CPU utilization, increase throughput, reduce response time, and ensure better scalability during peak shopping periods.

Conclusion

The e-commerce platform relies on efficient process management to handle millions of customer requests simultaneously. The operating system manages activities such as authentication, product search, payment processing, inventory updates, and shipment tracking as separate processes. **Process Control Blocks (PCB)** and **context switching** ensure smooth execution, while **multiprogramming** and efficient CPU scheduling improve system performance. By adopting **Multi-Level Feedback Queue Scheduling**, optimizing context switching, and implementing load balancing, the organization can achieve better CPU utilization, faster response times, improved scalability, and a seamless shopping experience during high-demand periods.

Question 8 – Process Scheduling in a Smart Hospital Management System

Scenario Overview

A smart hospital has implemented an automated healthcare management system to manage patient registration, diagnostic services, medical record access, pharmacy management, and emergency response operations. During emergency situations, doctors experience delays while accessing patient records because multiple hospital services compete for CPU resources simultaneously. As a result, emergency cases are not processed quickly, affecting patient care. To improve system performance, the hospital administration has requested the IT team to analyze different CPU scheduling algorithms and recommend the most suitable scheduling strategy.

Scheduling Challenges

The hospital management system faces several scheduling challenges due to the simultaneous execution of multiple services.

- Emergency services require immediate CPU access.
- Routine tasks such as pharmacy updates and patient registration also consume CPU resources.
- Multiple processes compete for the processor during peak hours.
- Increased waiting time delays access to critical patient records.
- High CPU contention reduces overall system performance.

Therefore, the operating system must prioritize emergency processes while ensuring that routine tasks are not permanently delayed.

Comparison of Scheduling Algorithms

Different CPU scheduling algorithms can be used in the hospital management system.

First Come First Serve (FCFS)

FCFS executes processes according to their arrival time. It is simple but unsuitable because emergency processes may wait behind routine tasks, increasing response time.

Shortest Job First (SJF)

SJF executes processes with the shortest burst time first. Although it reduces average waiting time, it does not consider the importance of emergency medical services.

Round Robin (RR)

Round Robin allocates equal CPU time to every process using a fixed time quantum. It ensures fairness but may delay emergency operations because all processes receive equal priority.

Priority Scheduling

Priority Scheduling assigns higher priority to emergency services and lower priority to routine hospital operations.

Process	Priority
Emergency Patient Care	High
Medical Record Access	High
Diagnostic Services	Medium
Pharmacy Management	Medium
Patient Registration	Low

Among these algorithms, **Priority Scheduling** is the most suitable because it ensures that emergency services receive immediate CPU access.

Recommended Scheduling Algorithm

The recommended scheduling algorithm is **Priority Scheduling** with the **Aging technique**.

Under this approach:

- Emergency processes receive the highest priority.
- Critical patient records are accessed immediately.
- Routine hospital services execute after emergency tasks.
- Aging gradually increases the priority of waiting processes, preventing starvation.

This method provides both fast emergency response and fairness for routine operations.

Performance Evaluation

Priority Scheduling significantly improves hospital system performance.

- Emergency patients receive faster medical attention.

- Response time for critical services is greatly reduced.
- CPU utilization improves because important processes execute without unnecessary delays.
- Throughput increases as more healthcare requests are completed efficiently.
- Patient safety and service quality improve during emergency situations.

Compared to FCFS, SJF, and Round Robin, Priority Scheduling provides the best performance for healthcare environments.

Future Improvements and Limitation

To support future expansion, the hospital can integrate **Priority Scheduling** with **Multi-Level Feedback Queue (MLFQ)** scheduling and cloud-based resource management. This enables the system to efficiently handle larger workloads during mass casualty incidents while maintaining fast emergency response.

The main limitation of Priority Scheduling is **starvation**, where low-priority tasks may wait for a long time if emergency cases continue to arrive. This issue can be resolved by implementing the **Aging** technique, which gradually increases the priority of waiting processes and ensures that every task eventually receives CPU time.

Conclusion

The smart hospital management system requires an efficient CPU scheduling strategy to prioritize emergency healthcare services while maintaining fairness for routine operations. After comparing FCFS, SJF, Round Robin, and Priority Scheduling, **Priority Scheduling with Aging** is identified as the most suitable solution. It provides faster response time, improves CPU utilization, increases throughput, reduces waiting time, and ensures timely access to critical patient records. By implementing this scheduling approach, the hospital can deliver reliable and efficient healthcare services even during emergency situations.

Question 9 – Inter-Process Communication (IPC) and Cooperating Processes

Scenario Overview

A smart city traffic management system integrates several independent subsystems, including **traffic signals**, **CCTV cameras**, **weather stations**, **public transportation services**, and **emergency vehicle tracking systems**. These subsystems continuously exchange information to monitor traffic conditions, control signal timings, and provide safe and efficient transportation. During periods of heavy traffic or emergencies, communication failures between these processes can delay traffic

signal updates and emergency vehicle movement, affecting public safety. Therefore, the operating system must provide an efficient **Inter-Process Communication (IPC)** mechanism to ensure reliable communication and coordination among all cooperating processes.

Analysis of Cooperating Processes

The smart city system consists of several cooperating processes that work together to achieve efficient traffic management.

- **Traffic Signal Control Process** – Controls traffic lights based on traffic conditions.
- **CCTV Monitoring Process** – Detects congestion, accidents, and traffic violations.
- **Weather Monitoring Process** – Collects weather information that may affect road conditions.
- **Public Transportation Process** – Monitors buses, trains, and other public transport services.
- **Emergency Vehicle Tracking Process** – Tracks ambulances, police, and fire vehicles for priority movement.

These processes continuously exchange information to ensure smooth traffic flow and quick emergency response.

Communication Requirements

Effective communication among these processes is essential for efficient city operations. For example, CCTV cameras must send accident information to the traffic signal controller so that signal timings can be adjusted immediately. Similarly, emergency vehicle tracking systems must communicate with traffic signals to provide a green corridor for ambulances and fire services.

The communication system should provide:

- Fast data exchange.
- Reliable message delivery.
- Process synchronization.
- Real-time updates.
- Secure communication.
- High system availability.

Without proper IPC, delays and communication failures may reduce the efficiency of the entire traffic management system.

Comparison of IPC Mechanisms

Two common IPC techniques are **Shared Memory** and **Message Passing**.

Shared Memory

Shared Memory allows multiple processes to access the same memory region.

Advantages

- Very fast communication.
- Suitable for exchanging large amounts of data.
- Low communication overhead.

Disadvantages

- Requires synchronization mechanisms.
- More complex to implement.
- Risk of data inconsistency if processes access memory simultaneously.

Message Passing

Message Passing enables processes to exchange information by sending and receiving messages through the operating system.

Advantages

- Better synchronization.
- Easier to implement.
- More secure communication.
- Suitable for distributed systems like smart cities.

Disadvantages

- Slightly slower than shared memory.
- Additional overhead due to message transfer.

Since the smart city consists of distributed components located at different locations, **Message Passing** is the most appropriate IPC mechanism.

Recommended IPC Mechanism

The recommended IPC technique is **Message Passing**.

Using Message Passing:

- Traffic signals receive accident alerts from CCTV cameras.
- Emergency vehicle tracking systems communicate with traffic controllers.
- Weather stations send weather updates to traffic management servers.
- Public transport systems exchange schedule information with central servers.

This approach ensures reliable communication, better synchronization, and easier management of distributed processes.

Fault Tolerance and Performance Improvement

To improve reliability and scalability, the operating system should implement:

- Message queues to prevent data loss.
- Process synchronization using semaphores or mutexes.
- Backup communication channels for fault tolerance.
- Secure encrypted communication between processes.
- Load balancing across multiple servers.
- Continuous monitoring of communication failures.

These improvements ensure uninterrupted communication even during hardware failures or network congestion.

Conclusion

The smart city traffic management system depends on effective communication among multiple cooperating processes such as traffic signals, CCTV cameras, weather stations, public transportation, and emergency vehicle tracking systems. After comparing Shared Memory and Message Passing, **Message Passing** is identified as the most suitable IPC mechanism because it provides reliable, secure, and synchronized communication in a distributed environment. By implementing message queues, synchronization techniques, secure communication, and fault-tolerant architecture, the operating system can improve system reliability, enhance traffic management, reduce emergency response time, and support future expansion of smart city infrastructure.

Question 10 – Threads and Multithreading Models

Scenario Overview

A global media company streams live sporting events to millions of viewers worldwide. To provide a smooth viewing experience, the application uses multiple threads for different tasks, including **video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion**. During high-demand events such as the **FIFA World Cup Final**, the number of users increases significantly. The software architects must analyze the current multithreading model and recommend a suitable solution to improve performance, scalability, and reliability.

Analysis of Threads and Multithreading Models

The streaming application uses several threads that execute concurrently.

- **Video Decoding Thread** – Decodes video frames for playback.
- **Audio Synchronization Thread** – Synchronizes audio with video.
- **Subtitle Generation Thread** – Displays subtitles in real time.
- **Buffering Thread** – Downloads and stores video data before playback.
- **Advertisement Thread** – Loads and displays advertisements during streaming.

These threads execute simultaneously, allowing different parts of the application to run independently and improving the overall user experience.

The operating system can implement different multithreading models:

Many-to-One Model

Multiple user threads are mapped to a single kernel thread. Although it is simple and efficient, if one thread blocks, the entire application is blocked. Therefore, this model is unsuitable for live streaming.

One-to-One Model

Each user thread is mapped to a separate kernel thread. This allows true parallel execution on multi-core processors and prevents one blocked thread from stopping other threads. However, creating a large number of kernel threads increases system overhead.

Many-to-Many Model

Multiple user threads are mapped to multiple kernel threads. This model combines the advantages of both previous models by providing better scalability, efficient resource utilization, and improved parallel execution.

Among these models, the **Many-to-Many Multithreading Model** is the most suitable for large-scale video streaming applications.

Recommended Multithreading Strategy

The operating system should implement the **Many-to-Many Multithreading Model** together with efficient thread synchronization.

The thread execution sequence is shown below.

User Request

|



Create Multiple Threads

|



Video | Audio | Subtitle | Buffer | Advertisement

|



Kernel Thread Pool

|



CPU Execution

|



Smooth Video Streaming This approach enables multiple threads to execute simultaneously while efficiently utilizing available CPU cores.

Performance Improvement and Limitation

Using the Many-to-Many model offers several benefits:

- Improves CPU utilization.
- Supports parallel execution on multi-core processors.
- Reduces video buffering and freezing.
- Improves audio-video synchronization.
- Increases throughput during peak traffic.

- Enhances scalability for millions of users.
- Provides better overall user experience.

To further improve synchronization, the application should use:

- Mutex locks for shared resources.
- Semaphores for thread synchronization.
- Thread pools to reduce thread creation overhead.
- Load balancing across multiple servers.
- Continuous monitoring of thread performance.

Limitation

The Many-to-Many model is more complex to implement than One-to-One or Many-to-One models. Managing multiple user threads and kernel threads requires additional operating system support and synchronization mechanisms. However, the performance and scalability benefits outweigh this complexity, especially for large-scale streaming platforms.

Conclusion

The global media streaming application relies on multiple threads to perform video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion simultaneously. After comparing the One-to-One, Many-to-One, and Many-to-Many models, the Many-to-Many model is the most suitable for large-scale streaming platforms. By combining this model with thread synchronization techniques, thread pools, and load balancing, the streaming platform can deliver uninterrupted, high-quality video streaming to millions of users even during major live sporting events.